

CO2 – AT3

DEBUGGING & OPTIMISATION TASK

REPORT:

Debugging and Correcting the Linear Search Algorithm

Aim

To identify and correct errors in the given Linear Search implementation, improve its readability and efficiency, and analyze its time complexity.

Problem Statement

The given Linear Search code fails to return correct results because the return statements are placed incorrectly.

Given Code

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return -1    # Error  
    return i            # Error
```

Errors Identified

Error 1

```
return -1
```

Problem

When the element is found, the function returns **-1**, which indicates that the element is **not found**.

Correct Statement

```
return i
```

This returns the index where the element is found.

Error 2

```
return i
```

Problem

This statement is outside the loop.

If the element is not found, the function returns the last value of **i**, giving an incorrect index.

Correct Statement

return -1

This correctly indicates that the element does not exist in the array.

Corrected Algorithm

1. Start.
2. Read the array and search key.
3. Traverse the array from the first element.
4. Compare each element with the key.
5. If a match is found, return the index.
6. If the loop finishes without finding the key, return **-1**.
7. Stop.

Corrected Source Code

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
    return -1  
  
arr = [10, 20, 30, 40, 50]  
key = int(input("Enter the element to search: "))  
result = linear_search(arr, key)  
if result != -1:  
    print("Element found at index:", result)  
else:  
    print("Element not found.")
```

Source Code Screenshot:



The screenshot shows a code editor with a file named 'main.py'. The code implements a linear search function. The output panel on the right shows the program's execution: it prompts for an element to search, receives '40', and outputs 'Element found at index: 3'. A success message '=== Code Execution Successful ===' is also displayed.

```
main.py
1- def linear_search(arr, key):
2-     for i in range(len(arr)):
3-         if arr[i] == key:
4-             return i
5-     return -1
6- arr = [10, 20, 30, 40, 50]
7- key = int(input("Enter the element to search: "))
8- result = linear_search(arr, key)
9- if result != -1:
10-     print("Element found at index:", result)
11- else:
12-     print("Element not found.")
```

Output

Enter the element to search: 40
Element found at index: 3
=== Code Execution Successful ===

Program Output

Case 1

Input

30

Output

Element found at index: 2

Case 2

Input

45

Output

Element not found.

Analysis

Working of Linear Search

The algorithm checks each element one by one until the required element is found.

For the array

[10, 20, 30, 40, 50]

Searching for **30**

Step Current Element Comparison Result

1	10	$10 == 30$	No
2	20	$20 == 30$	No
3	30	$30 == 30$	Yes → Return Index 2

Searching for **45**

Step Current Element Comparison Result

1	10	No	Continue
2	20	No	Continue
3	30	No	Continue
4	40	No	Continue
5	50	No	Return -1

Time Complexity

Best Case

The element is found at the first position.

$$T(n) = O(1)$$

Average Case

The element is found somewhere in the middle.

$$T(n) = O(n)$$

Worst Case

The element is at the last position or not present.

$$T(n) = O(n)$$

Space Complexity

The algorithm uses only one loop variable.

$$O(1)$$

Optimization

The corrected implementation is improved because:

- Returns the correct index when the element is found.
- Returns **-1** only when the element is absent.
- Uses a simple and readable loop.
- Stops immediately after finding the element, avoiding unnecessary comparisons.

Result

The errors in the original Linear Search implementation were successfully identified and corrected. The updated program now returns the correct index when the search element is found and -1 when it is not present. The algorithm works correctly for all test cases, has a best-case time complexity of $O(1)$, an average and worst-case time complexity of $O(n)$, and requires $O(1)$ auxiliary space.